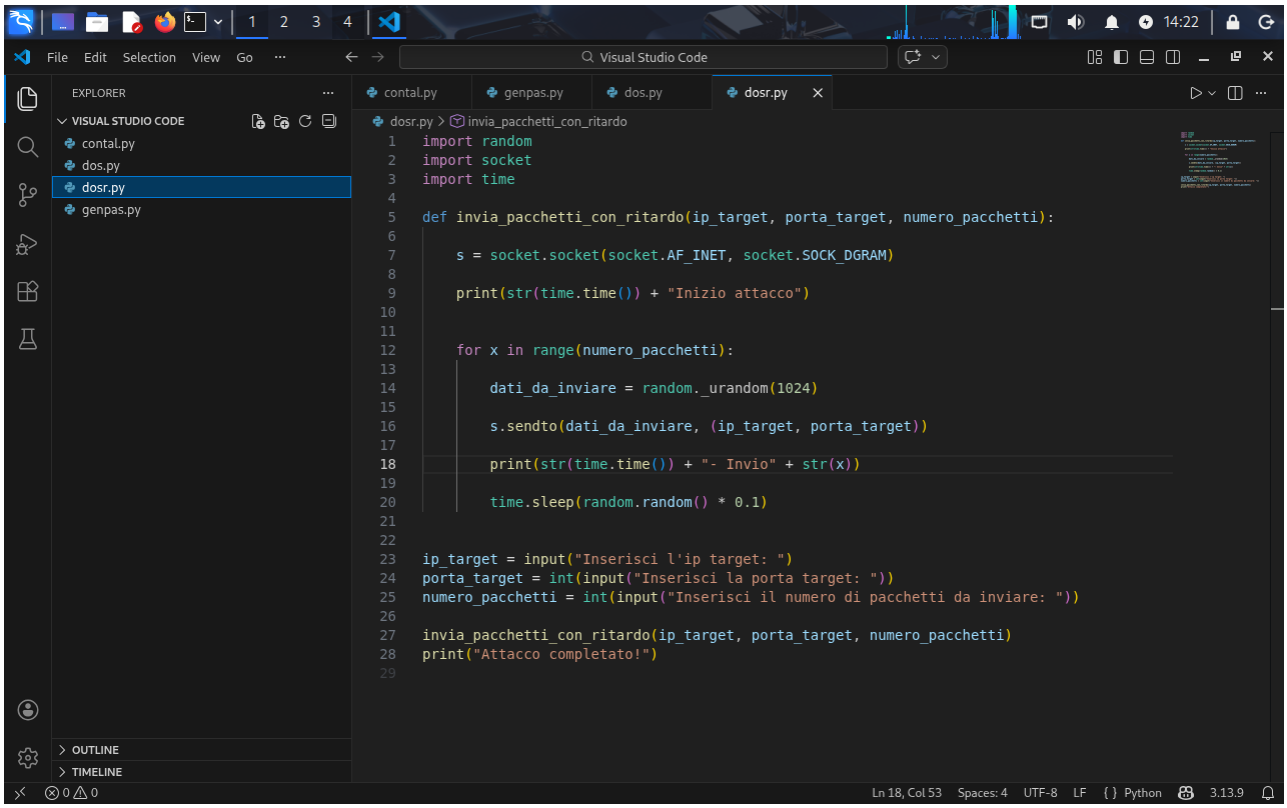
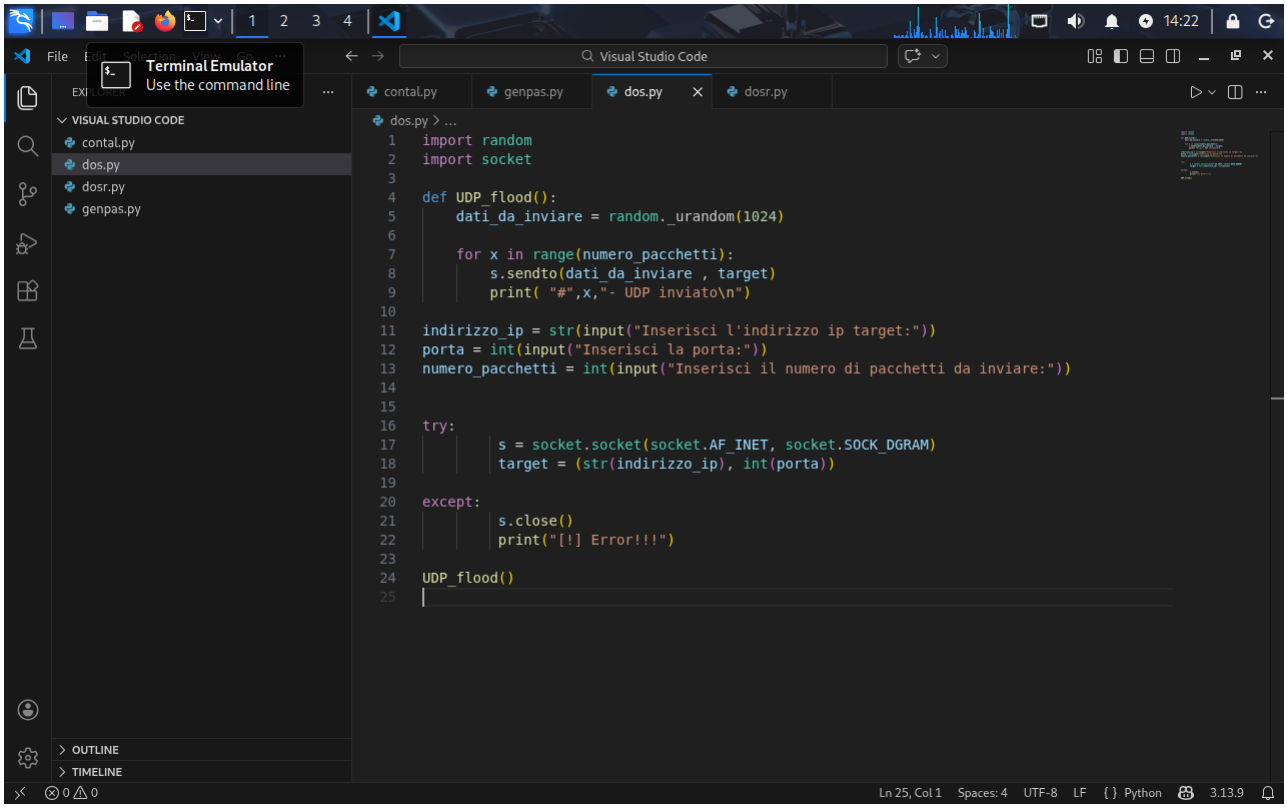
REPORT W7D4





Report – Simulazione di Invio Pacchetti UDP (Esercizio Base + Facoltativo)

1. Introduzione

In questo esercizio è stato sviluppato uno script in Python che utilizza la libreria socket per simulare l'invio di pacchetti UDP verso un host di destinazione.

L’obiettivo dell’attività è comprendere il funzionamento delle comunicazioni **UDP (User Datagram Protocol)** e osservare cosa accade quando vengono inviati molti pacchetti verso una stessa destinazione.

Il protocollo UDP è un protocollo di trasporto **senza connessione**, utilizzato quando è richiesta velocità di trasmissione piuttosto che affidabilità, come nei servizi di streaming o nei giochi online.

Gli esercizi svolti sono due:

- **Esercizio 1 (base):** invio rapido di pacchetti UDP
- **Esercizio 2 (facoltativo):** invio di pacchetti con ritardo casuale tra un invio e l’altro

---

2. Tecnologie utilizzate

Per la realizzazione degli script sono state utilizzate le seguenti tecnologie:

- **Python 3**
- Libreria socket (comunicazioni di rete)
- Libreria random (generazione dati casuali)
- Libreria time (gestione dei tempi di invio)
- **Visual Studio Code** come ambiente di sviluppo
- Sistema operativo **Linux (Kali Linux)**

---

3. Descrizione Esercizio 1 – Invio di pacchetti UDP

Il primo script implementa una funzione che invia un certo numero di pacchetti UDP verso un indirizzo IP e una porta specificata dall’utente.

Funzionamento generale

Il programma:

1. Richiede all’utente:
    - indirizzo IP di destinazione
    - porta di destinazione
    - numero di pacchetti da inviare
2. Crea un **socket UDP** utilizzando la libreria socket.
3. Genera un blocco di dati casuali di **1024 byte** tramite random.urandom().
4. Invia i pacchetti al target tramite la funzione sendto().

5. Mostra a schermo il numero progressivo dei pacchetti inviati.

Concetti di rete osservati

Durante l'esecuzione si possono osservare alcuni aspetti importanti:

- UDP non stabilisce una connessione preventiva
- i pacchetti vengono inviati senza conferma di ricezione
- il sistema di destinazione potrebbe scartare pacchetti se riceve traffico troppo veloce

Questo esercizio permette quindi di comprendere **il comportamento del protocollo UDP sotto carico di traffico**.

---

4. Descrizione Esercizio 2 (Facoltativo) – Invio con ritardo

Nel secondo script è stata implementata una versione migliorata del programma precedente.

La principale differenza consiste nell'introduzione di un **ritardo casuale tra l'invio dei pacchetti**.

Funzionamento

Il programma:

1. Richiede all'utente:
    - IP target
    - porta target
    - numero di pacchetti
2. Crea un socket UDP.
3. Per ogni pacchetto:
    - genera dati casuali
    - invia il pacchetto
    - stampa timestamp e numero del pacchetto
    - attende un intervallo di tempo casuale

Il ritardo viene generato tramite:

time.sleep(random.random() * 0.1)

che produce una pausa compresa tra **0 e 0.1 secondi**.

---

5. Differenze tra i due esercizi

Caratteristica	Esercizio 1	Esercizio 2
Invio pacchetti	continuo	con ritardo
Librerie	socket, random	socket, random, time
Controllo traffico	nessuno	pausa casuale
Monitoraggio	contatore pacchetti	timestamp + contatore

Il secondo script consente una simulazione più controllata del traffico di rete.

---

6. Considerazioni sulla sicurezza

Un invio massivo di pacchetti verso un sistema può causare:

- saturazione della banda
- sovraccarico del server
- rallentamenti o indisponibilità del servizio

Per questo motivo tali tecniche vengono studiate nei corsi di **cybersecurity** principalmente per:

- comprendere i rischi
- progettare sistemi di difesa
- implementare sistemi di mitigazione (firewall, rate limiting, IDS)

È importante utilizzare questi strumenti **solo in ambienti di laboratorio o su sistemi autorizzati**.

---

7. Conclusioni

L'attività ha permesso di comprendere:

- il funzionamento del protocollo UDP
- come Python può essere utilizzato per creare applicazioni di rete
- l'importanza del controllo del traffico nelle comunicazioni

Il secondo esercizio ha inoltre mostrato come l'introduzione di ritardi casuali possa modificare il comportamento del traffico generato.

Questi esperimenti sono utili per comprendere le dinamiche della rete e per studiare tecniche di protezione contro traffico anomalo.